

CONCLAVE: A Domain-Agnostic Multi-Agent Architecture for Real-Time Explainable Risk Detection

Abhishek Aditya

Independent

abhishek.aditya10@gmail.com

Abstract

Real-time risk detection—payment fraud, account takeover, intrusion—has historically been served by per-domain gradient-boosted tree models: accurate but opaque, with each domain re-implementing identical streaming and feature infrastructure. Recent LLM work attaches *post-hoc* rationales to such scores but leaves the underlying decision architecture unchanged. We present CONCLAVE, an open-source platform that makes *multi-agent deliberation* a first-class decision architecture: for every event, three specialized context agents—a feature summarizer, a behavioral baseliner (per-entity rolling embeddings in pgvector), and a graph reasoner (bounded Cypher templates over Neo4j)—assemble a structured evidence package, and a deliberating LLM judge emits a calibrated risk score in $[0, 1]$, a `{ALLOW, REVIEW, BLOCK}` verdict, ranked contributing factors, and a plain-language explanation. The same code base serves two very different domains—payment fraud and security/SOC—through *configuration* alone; only feature extractors and graph schemas differ. We describe the architecture (a Java/Spring Boot data plane with a Python/LangGraph control plane, communicating over gRPC), its calibration discipline, and its graceful degradation guarantees. On labeled synthetic adversarial streams scored end-to-end through the running stack with a commodity hosted judge (google/gemini-3.1-flash-lite via OpenRouter), CONCLAVE attains ROC-AUC 0.999 on fraud (332 decisions; precision 100.0% at zero false positives at the BLOCK threshold; 100.0% of attacks flagged at REVIEW-or-higher). On security it attains AUC 0.819, catching graph-structural attacks (lateral movement) while missing attacks whose discriminative signal no agent surfaces. That asymmetry is the paper’s central, honest lesson: *a deliberation architecture is only as good as the evidence its agents compute—the judge cannot recover a signal that was never extracted*. We release the full stack, generators, and benchmark harness.

Keywords

multi-agent systems, large language models, fraud detection, intrusion detection, explainable AI, stream processing, risk scoring

1 Introduction

A single event—a card-not-present charge, a login, a resource access—is rarely suspicious on its own. It becomes suspicious only in *context*: relative to the entity’s own past behavior, relative to the graph of entities it touches, and relative to known adversarial patterns. Production risk systems have answered this with per-domain gradient-boosted tree (GBT) models trained on hand-engineered features [1, 12]. These models are accurate and fast, but they have two persistent costs. First, they are *opaque*: a bare probability is hard for a fraud reviewer or SOC analyst to

act on, and the dominant remedy—post-hoc attribution such as SHAP [8] or LIME [11]—explains the model, not the decision, and is computed *after* the fact. Second, they are *redundant across domains*: payment fraud and security intrusion detection re-implement the same streaming, feature-store, and graph plumbing because the model and the features are entangled with the domain.

Large language models invite a different decomposition. Rather than bolting a rationale onto a tabular score, we can let specialized agents *reason over structured evidence* and have a judge issue both a calibrated score and a human-readable verdict in a single pass—explainable *by construction*, not after it. This is attractive precisely because the expensive parts of a risk pipeline—behavioral baselining, graph reasoning, calibration, audit—are domain-*independent*; only the feature definitions and the graph schema are domain-specific. If those are the only things that change, one architecture can serve many domains.

CONCLAVE is an engineering realization of that idea. It is a real-time platform that consumes an event stream and, per event, runs a four-agent *deliberation*:

- (1) a **feature agent** that distills the enriched event into a compact, structured digest;
- (2) a **behavioral baseliner** that compares the event to a per-entity rolling embedding (cosine similarity in pgvector);
- (3) a **graph reasoner** that runs depth-bounded Cypher templates over Neo4j to surface relational structure (rings, lateral movement);
- (4) a **deliberating judge** (an LLM) that weighs the assembled evidence and emits a calibrated Decision.

The same binaries serve two reference configurations—payment fraud and security/SOC—selected by an environment variable. Our contributions are:

- **An architecture pattern.** Multi-agent deliberation over a per-event evidence package as a first-class, domain-agnostic risk decision architecture, with a clean data-plane/control-plane split (Java/Spring Boot ↔ Python/LangGraph) and a pluggable judge provider (Anthropic, any OpenAI-compatible endpoint, or local Ollama).
- **An open-source artifact.** A complete, tested (219 Java + 99 Python tests), docker compose-deployable stack with two reference configurations, synthetic adversarial generators with leakage-proof ground-truth labels, and a reproducible benchmark harness.
- **An honest cross-domain evaluation.** End-to-end measurements on both domains, including a deliberate negative result—security exfiltration and account-takeover are missed—that we trace directly to *evidence coverage*, motivating a clear design principle for deliberation systems.

We frame this explicitly as an *engineering* paper: the contribution is the architecture and the artifact, not a novel learning algorithm. We believe the honesty of the negative security result is more useful to practitioners than a tuned headline number.

2 Background and Related Work

Tabular risk models and their explanations. Stripe’s Radar and comparable systems score card-not-present transactions with GBTs over hundreds of behavioral and network features at low latency [1, 12]. They are the right tool for raw discrimination, and we do not claim to beat them on AUC. Their interpretability gap is filled post-hoc with feature-attribution methods [8, 11]; these attribute a model’s output to its inputs but do not produce an analyst-facing rationale, and add latency. CONCLAVE instead makes the rationale the judge’s primary output.

Graph-based detection. Relational structure is central to both fraud (shared devices/cards across a ring) and intrusion (one identity touching many hosts). Graph neural networks are increasingly used for fraud [9], but they reintroduce the opacity problem and add training cost. CONCLAVE takes a deliberately simple stance: a fixed, audited set of depth-bounded Cypher templates whose outputs are *named* structured findings the judge can cite verbatim. This trades recall on novel structures for latency, predictability, and explainability.

Behavioral baselining. Per-entity baselines are classically hand-built statistical features (velocity counters, z-scores). We instead embed a textualized event sequence with a sentence-transformer [10] (all-MiniLM-L6-v2, 384-d) and keep a per-entity exponential-moving-average “fingerprint” in pgvector [6]; a new event is scored by cosine similarity to that fingerprint. As we show in §5, this signal is powerful for out-of-character spending but, by construction, blind to categorical attacks that embed as “normal”—a limitation the judge prompt is explicitly told about.

Multi-agent LLM systems. Frameworks such as AutoGen [14] and MetaGPT [4] orchestrate conversational agents for open-ended tasks, building on reasoning/acting patterns [13, 15]. CONCLAVE borrows the multi-agent decomposition but rejects free-form conversation: agents have strict tool budgets, the graph is a fixed DAG (parallel context-gathering then a single judge), and the output is a typed, calibrated record. The goal is a production decision under a latency budget, not an open-ended dialogue.

Calibration. Downstream consumers (step-up authentication, SOC triage queues) need a *score*, not a binary, and the score must mean something. Neural models are often miscalibrated [2]; we do not claim a theoretical calibration guarantee. Our calibration is *empirical*: a fixed threshold band ($\text{ALLOW} < 0.30 \leq \text{REVIEW} < 0.70 \leq \text{BLOCK}$) plus prompt-level anchoring guidance, validated against held-out labeled streams.

3 System Design

3.1 Event lifecycle

CONCLAVE is a streaming pipeline (Figure 1). A producer publishes raw events to a Kafka topic `events.{domain}.raw`. A Kafka Streams job enriches each event with computed features (velocity

counters, BIN risk, baseline-entity id, graph-entity ids) and emits `events.{domain}.enriched`. The decision orchestrator consumes the enriched stream, calls the deliberation service over gRPC, persists the resulting decision to Postgres, and re-emits it on `decisions.{domain}` for downstream consumers. An audit API and a live dashboard read from Postgres.

3.2 The deliberation graph

The deliberation itself is a small, fixed LangGraph DAG [7] (Figure 2). The feature node runs first and produces a structured digest. Two context nodes—the behavioral baseliner and the graph reasoner—then run *in parallel*; each has a strict budget of a single gRPC call to its backing service and a per-call deadline. Their findings join, and the judge node runs last. This shape is latency-optimal: the two slow-ish context lookups overlap, and only the judge (the LLM call) is on the critical path alone.

3.3 The evidence package

The judge never sees raw data; it sees a structured *evidence package*: (i) the feature digest (headline + bullets + numeric features carried through for quantitative reasoning); (ii) the behavioral finding—cosine similarity to the entity’s rolling embedding, a derived anomaly score, and a cold-start flag for entities with no history; and (iii) the graph finding—a *named* template result with a structural `risk_signal` in $[0, 1]$. Keeping the evidence structured (rather than free text) lets every node be unit-tested and lets the judge cite specific quantities.

Behavioral baseline (vectors, not statistics). Each entity carries a 384-dimensional “fingerprint”: an exponential moving average (EMA) of the embeddings of its past events, computed with a local sentence-transformer [10] and stored in pgvector [6]. A new event e is embedded and scored by cosine similarity to the entity’s fingerprint μ ; the anomaly score is the clipped, scaled cosine distance, $a(e) = \min(1, \max(0, 1 - \cos(e, \mu))/d_{\max})$. A naive high-decay EMA would pin a new entity near its very first event for many updates; we ramp the effective decay, $eff = \min(decay, 1 - 1/n)$ for event count n , so the profile behaves as a simple average for the first $\approx 1/(1 - decay)$ events and as a true EMA thereafter ($decay = 0.85$ by default, $\approx$ a 7-event memory). Scoring is strictly read-only: a suspicious event is compared against, but never folded into, the profile it is measured against, so an attack cannot poison its own baseline within a single decision.

Graph reasoner (named, bounded templates). Rather than generate Cypher with an LLM, the graph reasoner runs a fixed, audited set of depth-bounded templates and returns *named* findings the judge can cite verbatim (e.g. `fraud_card_testing_ring`, `security_lateral_movement`). Each template maps a structural count to a monotone risk curve: the card-testing template fires once a single device fans out across more than three cardholders; the lateral-movement template is silent below five distinct hosts, ramps from 0.60 at the five-host threshold, and crosses 0.80 (“confirmed”) as the fan-out widens, capped so the signal is never inflated past what the count supports. Fixed templates trade recall on novel structures for predictable latency (§5) and for the explainability of a named pattern.

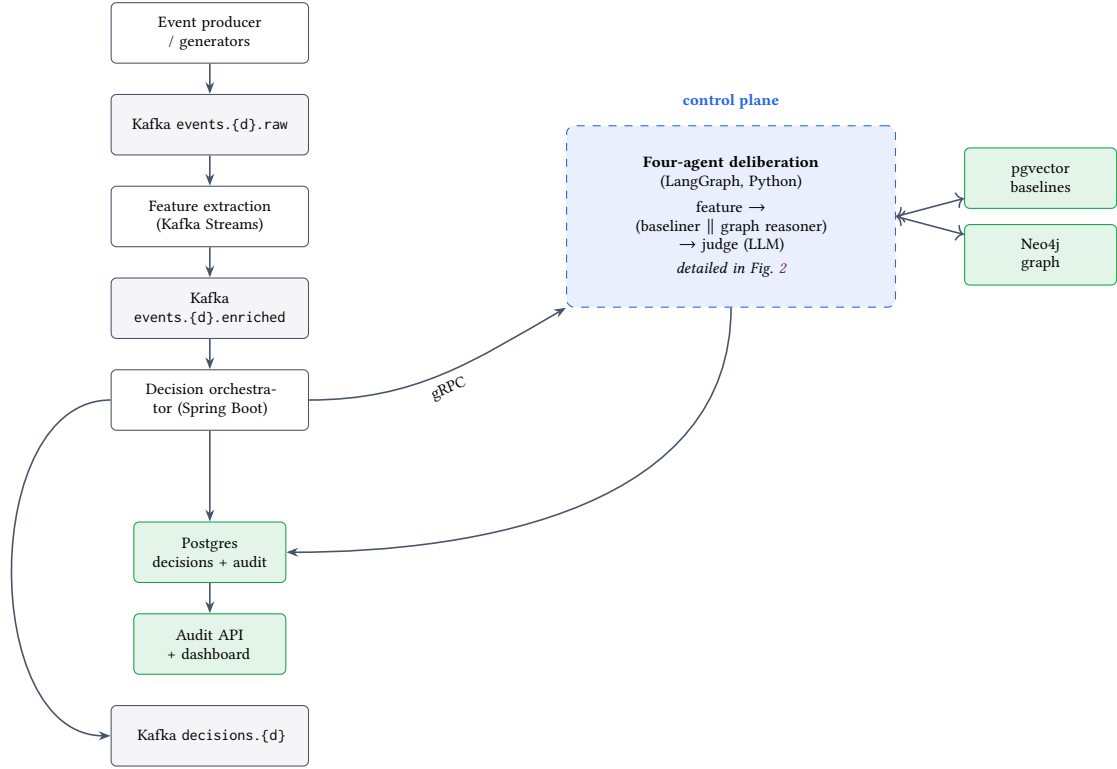


Figure 1: CONCLAVE event lifecycle. The *data plane* (Java/Spring Boot, left) handles ingestion, Kafka Streams feature extraction, the decision write path, and the audit API. The *control plane* (Python/LangGraph, centre) runs the four-agent deliberation behind a gRPC boundary; its two context agents query the backing stores (right). The internal deliberation DAG is shown in Figure 2. Only feature extraction and the graph schema are domain-specific.

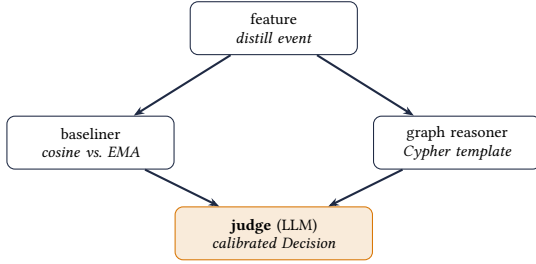


Figure 2: The deliberation DAG: $\text{feature} \rightarrow (\text{baseliner} \parallel \text{graph}) \rightarrow \text{judge}$. Any context node may fail or time out; its branch contributes an error note and the judge proceeds on the remaining evidence. If the judge itself is unavailable, a deterministic fallback derives a safe verdict from the structured findings.

3.4 Calibration and the judge contract

The judge is instructed to output a typed Decision: a score in $[0, 1]$, a verdict label from the fixed band, an ordered list of contributing factors with signed weights in $[-1, 1]$, and a 2–4 sentence Mark-down explanation. The calibration philosophy is encoded in the system prompt and is the most important design lever in the system:

anchor the score on the strongest piece of direct evidence—a severe rule indicator or a confirmed named graph pattern—and treat the behavioral embedding as the weakest, most easily fooled signal. The prompt explicitly warns that when an attacker repeats a malicious action, the entity’s rolling profile drifts toward the attack, so cosine similarity stays high (anomaly low) mid-attack; a “normal” embedding is weak corroboration, never exoneration. This guidance is what lets the judge BLOCK a card-testing ring even though the ring’s behavior looks self-consistent (Figure 4).

3.5 Graceful degradation

The pipeline never returns “no decision.” Each context node catches its own errors and degrades to an empty finding plus an error note; the parallel join concatenates errors without clobbering successful siblings. If the judge LLM is unavailable or returns an unparseable response, a deterministic fallback combines the structured signals (graph risk, cold-start prior, behavioral anomaly) into a safe, explainable verdict that leans toward REVIEW on ambiguity. In our benchmark runs, zero of 332+254 decisions hit the dead-letter queue.

3.6 Key design decisions

Several decisions follow from the goal of one architecture serving many domains under a latency budget, and are worth stating explicitly because they shaped the results in §5.

Table 1: Implementation footprint. LOC excludes generated code (Avro, protobuf stubs) and tests.

Component	LOC	Stack
Orchestrator	2,819	Spring Boot, Kafka
Baseline service	1,051	pgvector, MiniLM
Graph reasoner	1,108	Neo4j
Generators	1,723	Java
Deliberation agents	2,183	LangGraph, gRPC
Dashboard + site	2,548	React, Vite
Tests: 219 Java + 99 Python; CI gate $\geq 80\%$ line cov.		

- (1) **Java data plane, Python control plane.** The high-throughput Kafka consumer and the transactional decision-write path are Java; the stateful agent graph is Python/LangGraph. The gRPC seam keeps the “Java = data, Python = reasoning” separation legible and lets each side use its native ecosystem.
- (2) **Three context agents in parallel, one judge after.** Latency-optimal: the two backing-service lookups overlap, leaving only the LLM round-trip on the critical path alone (Figure 2).
- (3) **Vectors over hand-built statistics for baselines.** A rolling per-entity embedding captures “looks like this customer” more flexibly than a fixed bank of velocity/z-score features—at the documented cost that it is blind to categorical attacks (§5).
- (4) **Fixed Cypher templates, never LLM-generated queries.** Bounded depth gives predictable latency; *named* findings give the judge something concrete to cite; an audited template set is reviewable.
- (5) **A calibrated score, not a binary.** Downstream consumers (step-up auth, SOC triage) need a number; the verdict band is a view on the score, not a replacement for it.
- (6) **Pluggable judge behind one parser.** The provider factory makes the judge a swappable component, decoupling the architecture from any single vendor and enabling a fully local (Ollama) deployment.

4 Implementation

The data plane is Java 25 on Spring Boot 4.0 with Kafka Streams; the control plane is Python 3.12 on LangGraph. They communicate over gRPC. This split is deliberate: Java handles the high-throughput consumer and the transactional write path, while Python hosts the stateful agent graph where LangGraph’s parallel fan-out and conditional edges are natural. The judge backend is chosen at startup by an LLMProvider factory keyed on JUDGE_LLM_PROVIDER: anthropic (Claude via tool-use structured output, with prompt caching on the system block), openai (any OpenAI-compatible endpoint—OpenAI, OpenRouter, Groq—via JSON mode), or ollama (a local model via format=”json”). All three share one parser and one fallback, so the rest of the system is backend-agnostic. The behavioral embeddings run in-process via a Java sentence-transformer, so the *only* remote dependency that varies is the judge.

Table 2: One architecture, two configurations. Everything below the rule is shared verbatim; only the rows above it change per domain.

	Payment fraud	Security / SOC
Event	card-not-present txn	auth / resource access
Entity	cardholder / card	principal / identity
Graph	cardholder-device-IP -merchant	principal-host-resource
Patterns	card-testing ring, bust-out, ATO	lateral movement, exfiltration, ATO
<i>Shared:</i> deliberation DAG, evidence schema, EMA baseliner, calibration band, judge, Decision schema, audit API, dashboard, degradation path		

Domain configuration. Switching domains sets one environment variable (SPRING_PROFILES_ACTIVE). The same images then bind fraud-specific or security-specific feature specs, Avro schemas, and graph templates. No code is forked per domain; domain-specific logic lives only in the feature extractors and the Cypher templates, as Table 1’s shared components make concrete. Table 2 makes the mapping explicit: the two domains differ in their event type, entity, graph schema, and adversarial patterns, but share the deliberation graph, the calibration band, the judge, the decision schema, and the audit surface.

5 Evaluation

5.1 Methodology

We evaluate end-to-end through the running stack—no offline re-scoring. The synthetic generators emit a multi-day, multi-customer organic population (so that per-entity baselines converge before any attack), then inject labeled adversarial campaigns. Crucially, ground-truth labels are published to a *separate* Kafka topic (events.{domain}.labels) that the orchestrator never subscribes to, so there is no label leakage into the decision path. Our harness reads the persisted decisions from the audit API and joins them against the label topic on event_id.

The judge for these runs is google/gemini-3.1-flash-lite served via OpenRouter (an OpenAI-compatible endpoint). We report this configuration honestly: it is a commodity, low-cost hosted model, not the project’s canonical Claude Haiku backend. Because the architecture is provider-agnostic, the numbers here characterize *this* judge; a stronger judge would be expected to do at least as well. We use identical generator parameters across domains (8 entities $\times$ 7 days $\times$ 3 events/day organic population; 40 extra clean events; 2 ring/lateral, 2 ATO, 2 bust-out/exfiltration campaigns), yielding 332 fraud and 254 security decisions.

Every number below comes from a clean run of the published stack—three commands per domain (bring the stack up in serving mode, fire the labeled synthetic burst, capture):

```
./scripts/up.sh serving fraud
./scripts/seed.sh fraud --customers 8 --days 7 \
--events-per-day 3 --clean 40 --rings 2 --ato 2 --extra 2
python3 benchmark/capture.py --domain fraud --tag run
```


The harness pages the audit API for all decisions, drains the ground-truth label topic from Kafka, joins on event_id, and emits the confusion matrices, ROC-AUC, per-attack-type recall, score separation, and latency percentiles reported here. ROC-AUC uses the tie-aware Mann–Whitney formulation; precision@FPR sweeps unique score thresholds. The generator is seeded deterministically, so the event population is fixed; only the judge’s LLM responses vary run-to-run, and two independent runs reproduced the fraud AUC to within ± 0.0004 (measured 0.9985 and 0.9989) at the same zero-false-positive BLOCK operating point.

5.2 Detection: fraud

On the fraud configuration (Table 3), CONCLAVE attains ROC-AUC 0.999. The score cleanly separates classes: clean events average 0.16 while attacks average 0.88 (Figure 3, top). At the BLOCK threshold (≥ 0.70), precision is 100.0% with *zero* false positives (80 TP, 0 FP, 16 FN, 236 TN) and recall 83.3%. Widening the operating point to “flagged” (BLOCK *or* REVIEW) raises recall to 100.0%—*every* attack is surfaced for a human—at 80.7% precision and a 9.7% false-positive rate. At a fixed FPR $\leq 1\%$ the score achieves 98.9% precision and 93.8% recall. Per attack type, all three fraud campaigns—card-testing rings ($n=80$), account-takeover ($n=8$), and bust-out ($n=8$)—are flagged at 100% recall at the review-or-higher operating point.

5.3 Detection: security, and the evidence-coverage lesson

The security configuration tells a more interesting story (Table 3, lower block; Table 4). Overall AUC is 0.819, and the BLOCK threshold again yields 100.0% precision with zero false positives—but recall is only 30.6%. Breaking recall down by attack type explains why:

- **Lateral movement** ($n=16$): recall 69%. These attacks fan a single principal across many hosts, which the security_lateral_movement Cypher template detects directly; once the fan-out crosses the template’s host threshold, the judge BLOCKS with score 0.95 and cites the named pattern (Figure 4, lower). The misses are early events in a campaign, *before* enough hosts are touched for the structural signal to fire.
- **Exfiltration** ($n=12$): recall 0%. Every exfiltration event scored 0.15 (ALLOW).
- **Security account-takeover** ($n=8$): recall 0%. Scores clustered at 0.05–0.15 (ALLOW).

The exfiltration and ATO misses are not random: they are a direct, explainable consequence of *evidence coverage*. Exfiltration in our generator is a single principal reading many *sensitive resources* in sequence—a signal that a privileged-access template would surface, but the security deliberation’s hot path invokes only the lateral-movement template. With no host fan-out and no privileged-access finding, the judge sees a successful authentication, a “normal” embedding, and *nothing structural to act on*; it correctly (given its inputs) returns ALLOW. Security ATO is worse: its discriminative signal is a behavioral break (geo, time-of-day, auth method) that the same-entity embedding is, by the prompt’s own admission, blind to—when the attacker’s events accumulate, the profile drifts toward them.

Table 3: End-to-end detection. “Block” is the verdict ≥ 0.70 operating point; “Flagged” is BLOCK-OR-REVIEW (≥ 0.30). Judge: google/gemini-3.1-flash-lite.

Domain	Op. point	Prec.	Recall	FPR	AUC
Fraud (332)	Block	100.0%	83.3%	0.0%	0.999
	Flagged	80.7%	100.0%	9.7%	
Security (254)	Block	100.0%	30.6%	0.0%	0.819
	Flagged	91.7%	30.6%	0.5%	

Table 4: Per-attack-type recall at the flagged (review-or-higher) operating point. Detection tracks evidence coverage: patterns with a dedicated agent signal are caught; those without are missed.

Domain	Attack type	Surfacing signal	Recall
Fraud	Card-testing ring	graph template	100%
Fraud	Account-takeover	rule features	100%
Fraud	Bust-out	rule features	100%
Security	Lateral movement	graph template	69%
Security	Exfiltration	<i>none in hot path</i>	0%
Security	Account-takeover	<i>embedding blind</i>	0%

This is the paper’s central lesson, and we state it plainly: **a deliberation architecture is bounded by the evidence its agents compute. The judge is a weigher of presented evidence, not an oracle; it cannot recover a discriminative signal that no agent extracted.** The fix is not a better prompt—it is wiring the privileged-access template into the security hot path and adding a non-self-referential anomaly signal for ATO. We report the unpatched numbers deliberately, because the failure mode is the finding.

5.4 Latency

End-to-end deliberation latency is dominated by the LLM judge. The two context agents are fast and effectively free relative to the judge: the behavioral lookup’s measured p99 is 0.74 ms and the graph template’s p99 is 6 ms on a $\sim 100\text{K}$ -edge graph (component latency integration tests). The end-to-end per-decision latency, including the hosted LLM round-trip, is summarized in Table 5: fraud p50 1820 ms / p99 7940 ms, security p50 1827 ms / p99 8437 ms. The variance is the hosted endpoint’s, not the pipeline’s—the deterministic stages contribute under 10 ms combined. This locates the single optimization lever squarely at the judge: model choice, prompt-cache hit rate, and batching.

5.5 Cross-domain transfer

The headline domain-agnosticism claim holds at the architectural level: both configurations run on the same binaries, the same deliberation graph, the same calibration band, the same audit surface, and the same judge, differing only in feature specs and graph templates. The fraud configuration demonstrates that when the domain’s discriminative patterns are wired as agent signals, the

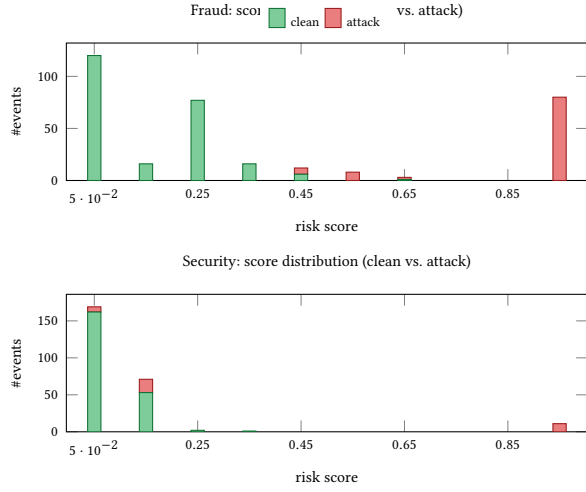


Figure 3: Calibrated-score histograms. In fraud (top) attacks concentrate near 1.0 and clean traffic near 0, with almost no overlap (AUC 0.999). In security (bottom) the caught attacks (lateral movement) sit at 0.95 but a mass of attacks (exfiltration/ATO) remains stuck in the low-score clean region—the visual signature of the evidence-coverage gap (AUC 0.819).

Table 5: End-to-end per-decision latency (ms), including the hosted judge round-trip. Deterministic stages (baseline p99 0.74 ms, graph p99 6 ms) are negligible; the judge dominates.

Domain	p50	p95	p99	max
Fraud	1820	3632	7940	8997
Security	1827	3101	8437	9138

shared architecture delivers production-grade detection (AUC 0.999, zero false-positive blocks). The security configuration demonstrates the converse and the boundary condition: the architecture transfers, but *detection quality transfers only as far as evidence coverage*.

5.6 Qualitative output

Figure 4 shows two real Decision records from the runs. The fraud example BLOCKS a card-testing ring with a precise, analyst-actionable rationale and signed contributing factors, including an explicit note that the “normal” behavioral signal is disregarded as an artifact of repetitive automated testing—exactly the calibration discipline from §3. The security example shows the same reasoning pattern catching lateral movement. These are verbatim outputs, not curated paraphrases.

6 Discussion and Limitations

The judge is a bottleneck—of latency and of evidence. Two senses. Operationally, the LLM dominates latency (Table 5), so throughput scales with judge concurrency and cache hit rate, not with the data plane. Epistemically, the judge is bounded by the evidence package; the security result shows that no amount of judging recovers an

```
// FRAUD -- card-testing ring, score 0.95 -> BLOCK
"verdict_explanation_md":
  "This transaction is part of a confirmed
  `fraud_card_testing_ring` involving 8 distinct
  cardholders sharing a single device. The graph
  reasoning signal is decisive (risk_signal=1.00),
  and the behavioral baseline's lack of anomaly is
  disregarded as it reflects the attacker's
  consistent, repetitive testing behavior.",
  "contributing_factors": [
    {"name": "graph_ring_detected", "weight": 1.0, ...},
    {"name": "no_anomaly_observed", "weight": -0.1, ...}]

// SECURITY -- lateral movement, score 0.95 -> BLOCK
"verdict_explanation_md":
  "The event is part of a confirmed
  `security_lateral_movement` graph pattern ...
  accessing 8 distinct hosts in a suspicious
  sequence. Despite the high cosine similarity,
  this behavioral signal is disregarded ... The
  graph evidence is decisive."
```

Figure 4: Verbatim judge outputs (abridged). Both domains share the same decision shape and the same calibration discipline—anchor on the confirmed graph pattern, discount the self-consistent embedding.

un-extracted signal. Both point to the same engineering priority: invest in *agent coverage* and in judge serving, in that order.

Calibration is empirical, not theoretical. The threshold band and the score anchoring are validated against labeled streams, not derived from a calibration guarantee [2]. A different judge model will shift the score distribution; the band would need re-validation. We do not claim transferable calibration across judge backends.

Synthetic evaluation. Our adversarial streams are generated, not production traffic, so absolute numbers should be read as characterizing the architecture under known, labeled conditions—not as a claim about real-world IEEE-CIS [5] or BETH [3] performance, which we leave to future work. The generators do encode realistic structure (multi-day baselines, mixed amount distributions, ring/lateral fan-out), and the leakage-proof label topic makes the join trustworthy.

No comparison to a tuned GBT baseline. We deliberately do not claim to beat XGBoost on AUC; a well-tuned GBT will likely win on raw discrimination. CONCLAVE’s claim is architectural: explainability by construction, domain-agnosticism, and graceful degradation, at competitive detection *when evidence coverage is complete*. A head-to-head on public datasets is the natural next study.

Single-judge, single-tenant. The judge is one LLM call, not an ensemble or a debate; we found a single well-prompted judge over good evidence sufficient and cheaper. The system is single-tenant by design scope.

7 Conclusion

CONCLAVE treats multi-agent deliberation as a first-class risk-decision architecture: specialized agents assemble a structured, per-event evidence package and a calibrated LLM judge issues an explainable verdict, with the same code serving payment fraud and security through configuration alone. End-to-end, the fraud configuration reaches AUC 0.999 with zero false-positive blocks

and flags every injected attack; the security configuration cleanly catches graph-structural attacks and—honestly—misses attacks whose signal no agent surfaces. That asymmetry crystallizes the design principle we most want practitioners to take away: *deliberation quality is upper-bounded by evidence coverage*. Future work is concrete and architecture-internal: wire the remaining graph templates into the security hot path, add a non-self-referential anomaly signal for account-takeover, validate on IEEE-CIS and BETH, and benchmark against a tuned GBT baseline. The complete stack, generators, benchmark harness, and the $\LaTeX$ source of this paper are open-source at <https://github.com/Abhishek-Aditya-bs/Conclave>.

References

- [1] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794.
- [2] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning (ICML)*.
- [3] Kate Highnam, Kai Arulkumaran, Zachary Hanif, and Nicholas R. Jennings. 2021. BETH Dataset: Real Cybersecurity Data for Anomaly Detection Research. In *ICML Workshop on Uncertainty and Robustness in Deep Learning*.
- [4] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *International Conference on Learning Representations (ICLR)* (2024). arXiv:2308.00352.
- [5] IEEE Computational Intelligence Society and Vesta Corporation. 2019. IEEE-CIS Fraud Detection. <https://www.kaggle.com/c/ieee-fraud-detection>.
- [6] Andrew Kane. 2024. pgvector: Open-source Vector Similarity Search for Postgres. <https://github.com/pgvector/pgvector>.
- [7] LangChain. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. <https://langchain-ai.github.io/langgraph/>. Accessed 2026-05.
- [8] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)* (2017).
- [9] Soroosh Motie and Bijan Raahemi. 2024. Financial Fraud Detection Using Graph Neural Networks: A Systematic Review. *Expert Systems with Applications* 240 (2024).
- [10] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2019).
- [11] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2016).
- [12] Stripe. 2023. How Radar Works: Machine Learning for Fraud Detection. <https://stripe.com/radar/guide>. Accessed 2026-05.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)* (2022).
- [14] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155.
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.